CITY CARE

Documentación completa de asignación a dependencias y equipos

Etapa 3 4 del módulo de reportes e incidencias

La etapa 3.4 incorpora la organización operativa de las incidencias. Cada dependencia puede administrar equipos con líder e integrantes, asignar reportes con prioridad y fecha límite, reasignarlos sin perder trazabilidad y retirar una asignación con un motivo verificable.

Indicador	Definición
Estado	Completo y validado
Rutas nuevas	9 operaciones administrativas
Tablas nuevas	work_teams, work_team_members y report_assignments
Migraciones totales	15 aplicadas en MySQL
Endpoints totales	31 rutas registradas
Suite funcional	64 de 64 pruebas aprobadas
Contrato	OpenAPI y cliente Orval sincronizados en el puerto 3333

Resultado de la etapa

El backend permite crear y mantener equipos por dependencia, designar al líder, reemplazar integrantes y controlar la disponibilidad del equipo. Los reportes admiten una sola asignación activa, mientras el historial conserva cada asignación, reasignación y retiro con actor, fechas, prioridad y observaciones.

1 Objetivo y alcance

- Crear equipos de trabajo asociados a una dependencia activa.
- Designar un líder y mantener una lista de integrantes activos sin duplicados.
- Restringir la administración local a la dependencia del usuario autenticado.
- Asignar reportes a equipos de la misma dependencia.
- Registrar prioridad, fecha límite y observaciones operativas.
- Conservar el historial al reasignar o retirar una incidencia.
- Impedir la baja de equipos con asignaciones activas.
- Impedir la baja de usuarios mientras pertenezcan a un equipo activo.
- Publicar las operaciones en OpenAPI y regenerar el SDK Orval.
- Validar permisos, integridad y transacciones mediante pruebas funcionales.

2 Arquitectura funcional

Componente	Responsabilidad
Panel administrativo	Gestiona equipos y asigna incidencias mediante el SDK Orval.
Middleware auth	Obtiene al actor desde el token Bearer.
Middleware permission	Autoriza teams y reports.assign según el rol.
WorkTeamsController	Aplica alcance, validaciones, transacciones e historial.
AuthorizationService	Define permisos para admin y dependency_admin.
Lucid	Modela equipos, integrantes, asignaciones y relaciones.
MySQL	Conserva el estado actual y la trazabilidad histórica.
OpenAPI y Orval	Exponen nueve funciones tipadas para el frontend.

La autorización se aplica en dos niveles. El middleware comprueba el permiso y el controlador vuelve a comprobar dependency_id. Esta segunda validación evita que una cuenta con permiso administrativo opere sobre equipos o reportes de otra dependencia.

3 Modelo de datos

Tabla	Relación	Finalidad
work_teams	dependency_id y leader_user_id	Identidad, líder y estado del equipo.
work_team_members	work_team_id y user_id	Lista vigente de integrantes sin duplicados.
report_assignments	report_id y work_team_id	Historial operativo de asignaciones.
reports.current_assignment_id	report_assignments.id	Puntero a la única asignación activa.

El historial no se sobrescribe. Cuando una incidencia cambia de equipo, la asignación anterior recibe unassigned_at, unassigned_by_user_id y unassignment_note; después se crea un registro nuevo y reports actualiza current_assignment_id dentro de la misma transacción.

4 Tabla work teams

Campo	Tipo	Regla
id	entero	Identificador interno.
dependency_id	entero FK	Dependencia propietaria con eliminación restringida.
name	texto 150	Único dentro de la dependencia.
description	texto nullable	Descripción operativa opcional.
leader_user_id	entero FK nullable	Líder del equipo; SET NULL protege la integridad.
is_active	booleano	Baja lógica del equipo.
created_at y updated_at	fecha y hora	Auditoría temporal del registro.

5 Integrantes y asignaciones

5 1 Tabla work team members

Campo	Uso
work_team_id	Equipo relacionado; CASCADE elimina sus vínculos.
user_id	Integrante activo de la misma dependencia.
created_at	Fecha de incorporación vigente.
Índice único	Impide repetir un usuario dentro del mismo equipo.

5 2 Tabla report assignments

Grupo	Campos	Uso
Identidad	id, report_id, work_team_id	Reporte y equipo de cada movimiento.
Asignación	assigned_by_user_id, assigned_at	Actor y momento de asignación.
Planeación	priority, due_at, observations	Urgencia, compromiso y notas.
Retiro	unassigned_at, unassigned_by_user_id	Cierre de la asignación.
Motivo	unassignment_note	Explicación de retiro o reasignación.
Auditoría	created_at, updated_at	Fechas técnicas del registro.

6 Migraciones

Migración	Resultado
1796000000500 create work teams table	Crea equipos, líder, estado, índices y nombre único por dependencia.
1796000000600 create work team members table	Crea integrantes y evita duplicados por equipo.
1796000000700 create report assignments table	Crea el historial y agrega current_assignment_id a reports.

Las tres migraciones se aplicaron en el lote 8 de MySQL. El proyecto registra quince migraciones completed.

7 Ciclo de vida de un equipo

1. El administrador selecciona una dependencia activa.
2. El backend valida que el líder y los integrantes estén activos y pertenezcan a esa dependencia.
3. El líder se incorpora automáticamente a la lista de integrantes.
4. El equipo puede editar nombre, descripción, líder e integrantes.
5. Para cambiar integrantes se reemplaza la lista vigente en una operación controlada.
6. Un usuario no puede desactivarse mientras pertenezca a un equipo activo.
7. Un equipo con incidencias activas no puede desactivarse.
8. Después de retirar sus asignaciones puede aplicarse la baja lógica con is_active false.

8 Flujo de asignación

1. El administrador abre un reporte que ya tiene dependency_id.
2. Selecciona un equipo activo de la misma dependencia.
3. Define prioridad, observaciones y una fecha límite futura opcional.
4. El controlador bloquea el reporte durante la transacción.
5. Si existe una asignación activa, la cierra como reasignación administrativa.
6. Crea el nuevo registro de report_assignments.
7. Actualiza reports.current_assignment_id con el nuevo identificador.
8. Confirma ambos cambios juntos y devuelve el detalle de la asignación.

9 Endpoints del punto 3 4

Método	Ruta	Permiso	Función
GET	/api/v1/admin/teams	teams.list	Lista equipos con filtros.
POST	/api/v1/admin/teams	teams.create	Crea equipo y miembros.
GET	/api/v1/admin/teams/:id	teams.read	Consulta el detalle.
PATCH	/api/v1/admin/teams/:id	teams.update	Edita datos o líder.
PATCH	/api/v1/admin/teams/:id/status	teams.status	Aplica baja lógica.
PUT	/api/v1/admin/teams/:id/members	teams.members	Reemplaza integrantes.
POST	/api/v1/admin/reports/:id/assignments	reports.assign	Asigna o reasigna.
GET	/api/v1/admin/reports/:id/assignments	reports.read	Devuelve el historial.
DELETE	/api/v1/admin/reports/:id/assignment	reports.assign	Retira la asignación.

10 Filtros de equipos

Parámetro	Regla
page	Entero positivo; valor predeterminado 1.
perPage	De 1 a 100; valor predeterminado 20.
search	Busca en nombre o descripción.
dependencyId	Solo el administrador global puede elegir otra dependencia.
isActive	Filtra equipos activos o inactivos.

11 Ejemplos de peticiones

11 1 Crear un equipo

```
POST /api/v1/admin/teams
Authorization: Bearer <token>
Content-Type: application/json

{
  "dependencyId": 4,
  "name": "Cuadrilla nocturna",
  "description": "Atención de luminarias",
  "leaderUserId": 18,
  "memberUserIds": [18, 21, 24]
}
```

El arreglo no puede contener identificadores duplicados. Si el líder no aparece en memberUserIds, el backend lo agrega automáticamente.

11 2 Asignar una incidencia

```
POST /api/v1/admin/reports/42/assignments
Authorization: Bearer <token>
Content-Type: application/json

{
  "workTeamId": 7,
  "priority": "high",
  "dueAt": "2026-11-05T18:00:00.000Z",
  "observations": "Atender antes del fin de semana"
}
```

dueAt debe ser una fecha ISO válida y futura. observations admite de 3 a 2000 caracteres cuando se envía.

12 Reasignación y retiro

```
DELETE /api/v1/admin/reports/42/assignment
Authorization: Bearer <token>
Content-Type: application/json

{
  "note": "La incidencia será atendida por otra cuadrilla"
}
```

El retiro exige un motivo de 3 a 2000 caracteres. En una reasignación, el sistema usa el motivo interno Reasignación administrativa y crea inmediatamente la nueva asignación.

13 Prioridades y fechas

Código	Interpretación operativa	Uso recomendado
low	Baja	Incidencia sin riesgo inmediato.
medium	Media	Atención dentro de la planeación normal.
high	Alta	Afectación relevante que requiere atención pronta.
urgent	Urgente	Riesgo o afectación crítica.

La prioridad no cambia automáticamente el estado del reporte. La asignación organiza el trabajo y el flujo de estados de 3.3 sigue registrando pending, in_progress y resolved de manera independiente.

14 Permisos y alcance

Operación	admin	dependency_admin	user
Listar y consultar equipos	Todas	Su dependencia	No
Crear y editar equipos	Todas	Su dependencia	No
Cambiar integrantes o estado	Todas	Su dependencia	No
Asignar reportes	Todas	Su dependencia	No
Consultar historial	Todas	Su dependencia	No

Los permisos agregados son reports.assign, teams.list, teams.read, teams.create, teams.update, teams.status y teams.members. El rol user no recibe ninguno de ellos.

15 Reglas de integridad

- La dependencia debe estar activa al crear el equipo.
- El nombre del equipo es único dentro de su dependencia.
- El líder y los integrantes deben estar activos y pertenecer a la misma dependencia.
- El líder siempre forma parte de los integrantes.
- El reporte debe tener una dependencia antes de asignarse.
- El equipo y el reporte deben compartir dependency_id.
- Solo puede existir una asignación activa señalada por current_assignment_id.
- Una fecha límite debe ser futura.
- Un equipo inactivo o sin personal disponible no puede recibir incidencias.
- Una cuenta ligada a un equipo activo no puede desactivarse hasta retirarla.

16 Estructura de la respuesta

```
{
  "data": {
    "id": 12, "reporteId": 42, "prioridad": "high",
    "equipo": {
      "id": 7, "nombre": "Cuadrilla nocturna",
      "lider": { "id": 18, "nombre": "Ana López", "email": "ana@example.com" },
      "integrantes": []
    },
    "asignadoEn": "2026-11-01T18:00:00.000Z",
    "fechaLimite": "2026-11-05T18:00:00.000Z",
    "activa": true
  }
}
```

Los listados y detalles administrativos de reportes incluyen asignacionActual cuando existe. El historial de asignaciones se consulta mediante su endpoint específico y se devuelve de la más reciente a la más antigua.

17 Errores controlados

HTTP	Código	Causa
403	E_PERMISSION_REQUIRED	La cuenta no posee el permiso.
403	E_DEPENDENCY_SCOPE	Operación fuera de la dependencia.
403	E_ASSIGNMENT_DEPENDENCY_SCOPE	Equipo y reporte pertenecen a dependencias distintas.
404	E_WORK_TEAM_NOT_FOUND	Equipo inexistente.
404	E_REPORT_NOT_FOUND	Reporte inexistente.
409	E_WORK_TEAM_DUPLICATE	Nombre repetido dentro de la dependencia.
409	E_WORK_TEAM_INACTIVE	El equipo está inactivo.
409	E_WORK_TEAM_UNAVAILABLE	Falta líder o hay integrantes inactivos.
409	E_WORK_TEAM_HAS_ACTIVE_ASSIGNMENTS	El equipo aún tiene incidencias.
409	E_USER_HAS_ACTIVE_TEAM	El usuario sigue dentro de un equipo activo.
409	E_REPORT_NOT_ASSIGNED	No existe asignación activa para retirar.
422	E_WORK_TEAM_MEMBERS_INVALID	Integrantes duplicados, inactivos o externos.
422	E_ASSIGNMENT_PRIORITY_INVALID	Prioridad fuera del catálogo.
422	E_ASSIGNMENT_DUE_DATE_INVALID	Fecha inválida, vencida o actual.

18 OpenAPI y Orval

El contrato openapi/city-care.v1.yaml publica las nueve operaciones de 3.4 y conserva el servidor http://127.0.0.1:3333. Orval validó el contrato y regeneró sdk/generated/city-care.ts sin referencias al puerto temporal 3334.

Operación	Función Orval
listWorkTeams	listWorkTeams
createWorkTeam	createWorkTeam
getWorkTeam	getWorkTeam
updateWorkTeam	updateWorkTeam
updateWorkTeamStatus	updateWorkTeamStatus
updateWorkTeamMembers	updateWorkTeamMembers
assignReport	assignReport
listReportAssignments	listReportAssignments
unassignReport	unassignReport

El SDK es generado y no debe editarse manualmente. El flujo correcto es actualizar controladores e interfaces, exportar OpenAPI desde el backend configurado en 3333 y ejecutar npm run openapi:generate.

19 Componentes implementados

Archivo	Responsabilidad
work_teams_controller.ts	CRUD, miembros, alcance, asignación, historial y retiro.
work_team.ts	Relaciones de dependencia, líder, integrantes y asignaciones.
report_assignment.ts	Relaciones del historial con reporte, equipo y actores.
work_team.ts validator	Filtros, cuerpos, tamaños y formatos.
authorization_service.ts	Matriz de permisos de 3.4.
admin_users_controller.ts	Bloquea bajas de integrantes de equipos activos.
report_presenter_service.ts	Incluye la asignación actual en reportes.
start/routes.ts	Registra las nueve rutas y normaliza OpenAPI.
handler.ts	Traduce errores operativos a respuestas uniformes.

20 Pruebas y evidencia

Comprobación	Resultado
Creación	Líder incluido automáticamente como integrante.
Integrantes	Duplicados, inactivos y externos rechazados.
Permisos	El ciudadano recibe 403 en administración de equipos.
Alcance	dependency_admin no puede salir de su dependencia.
Asignación	Prioridad, fecha y observaciones persistidas.
Reasignación	La anterior se cierra y el historial conserva ambas.
Bajas	Equipos e integrantes activos quedan protegidos.
Retiro	Exige motivo y libera current_assignment_id.
Suite completa	64 de 64 pruebas aprobadas.
TypeScript	npm run typecheck sin errores.
ESLint	npm run lint sin errores.
Build	Compilación de producción correcta.
OpenAPI y Orval	Contrato válido y SDK regenerado en 3333.
MySQL	15 migraciones completed.

21 Guía de comprobación manual

1. Iniciar MySQL y el backend en http://127.0.0.1:3333.
2. Iniciar sesión como admin o dependency_admin.
3. Crear dos usuarios activos dentro de la misma dependencia.
4. Crear un equipo con uno como líder y ambos como integrantes.
5. Consultar el equipo y confirmar líder, integrantes y dependencia.
6. Seleccionar un reporte de esa dependencia y asignarlo con prioridad high.
7. Consultar el historial y confirmar que la asignación está activa.
8. Reasignar a otro equipo y comprobar que el registro anterior queda cerrado.
9. Retirar la asignación actual enviando una nota.
10. Desactivar el equipo después de confirmar que no tiene incidencias activas.

22 Diagnóstico de problemas

Síntoma	Causa probable	Revisión
403 al crear	Dependencia fuera del alcance.	Comparar dependency_id del token.
422 en miembros	Usuario externo, inactivo o duplicado.	Revisar estado, dependencia e IDs.
409 al asignar	Equipo inactivo o sin personal disponible.	Consultar detalle del equipo.
403 al asignar	Equipo y reporte de dependencias distintas.	Comparar ambas dependencias.
422 en fecha	dueAt no es futura o no usa ISO.	Enviar una fecha ISO posterior.
409 al desactivar equipo	Mantiene una incidencia activa.	Retirar o reasignar primero.
409 al desactivar usuario	Continúa en un equipo activo.	Actualizar integrantes primero.
Orval apunta a otro puerto	OpenAPI se exportó con otra APP_URL.	Confirmar PORT 3333 y regenerar.

23 Seguridad y decisiones técnicas

- El actor siempre procede del token y nunca del cuerpo de la petición.
- El permiso se combina con una comprobación explícita de dependencia.
- Las reasignaciones usan bloqueo de fila y una sola transacción.
- La referencia current_assignment_id evita dos asignaciones vigentes para el mismo reporte.
- La baja lógica conserva equipos e historial para consultas futuras.
- Las llaves foráneas usan RESTRICT, CASCADE o SET NULL según el valor histórico del dato.
- Los logs registran identificadores operativos sin tokens ni credenciales.

24 Cierre de la etapa

La etapa 3.4 queda concluida. City Care ya puede estructurar cuadrillas por dependencia, asignar incidencias, definir prioridad y fechas, reasignar sin perder historial y proteger la separación administrativa. La base queda preparada para el punto 3.5, donde la inteligencia artificial propondrá una clasificación y la dependencia responsable sin reemplazar la revisión administrativa.